Database Schema for Aura AI (image genrator)

Tables:

1. **users** - Stores user details.
2. **images** - Stores generated images along with metadata.
3. **user_gallery** - Links users to their generated images.
4. **admin** - Stores admin credentials and access rights.
5. **password_resets** - Stores password reset tokens.

1. users Table (Stores user details)

```
CREATE TABLE users (
    id INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Columns:

- id → Unique identifier for each user (Primary Key).
- username → Unique username for the user.
- email → Unique email address.
- password_hash → Hashed password for security.
- created_at → Timestamp of account creation.

2. images Table (Stores generated images)

```
CREATE TABLE images (
    id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
    image_url VARCHAR(255) NOT NULL,
    prompt TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

Columns:

- id → Unique identifier for each image (Primary Key).
- user_id → The ID of the user who generated the image (Foreign Key).

- image_url → URL/location of the generated image.
- prompt → Text prompt used to generate the image.
- created_at → Timestamp of when the image was created.
- Cascade Delete: If a user is deleted, their images are automatically deleted.

---

3. user_gallery Table (Manages user image galleries)

```
CREATE TABLE user_gallery (
    id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
    image_id INT NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (image_id) REFERENCES images(id) ON DELETE CASCADE
);
```

Columns:

- id → Unique identifier for each gallery entry (Primary Key).
- user_id → The ID of the user who owns this gallery entry (Foreign Key).
- image_id → The ID of the image in the gallery (Foreign Key).
- Cascade Delete: If a user is deleted, their gallery entries are deleted too.

---

4. password_resets Table (Stores password reset requests)

```
CREATE TABLE password_resets (
    id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
    reset_token VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    expires_at TIMESTAMP NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

Columns:

- id → Unique identifier for each reset request (Primary Key).
- user_id → The ID of the user requesting a password reset (Foreign Key).
- reset_token → A unique token for password reset.
- created_at → Timestamp when the reset request was created.

- expires_at → Expiry time for the token (e.g., 1 hour after creation).
- **Cascade Delete:** If a user is deleted, their password reset requests are removed.

---

5. admin Table (Stores admin details)

```
CREATE TABLE admin (
    id INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL
);
```

Columns:

- id → Unique identifier for each admin (Primary Key).
- username → Unique admin username.
- email → Unique admin email.
- password_hash → Hashed password for security.

1. Retrieve All Users

```
SELECT * FROM users;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

id	username	email	password_hash	created_at
1	user1	user1@example.com	hashed_password1	2025-02-28 09:32:16
2	user2	user2@example.com	hashed_password2	2025-02-28 09:32:16
NULL	NULL	NULL	NULL	NULL

2. Retrieve All Images

```
SELECT * FROM images;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	user_id	image_url	prompt	created_at
1	1	https://example.com/image1.png	A futuristic cityscape at night	2025-02-28 09:32:23
2	2	https://example.com/image2.png	A cyberpunk warrior standing on a rooftop	2025-02-28 09:32:23
NULL	NULL	NULL	NULL	NULL

3. Retrieve All User Galleries

SELECT * FROM user_gallery;

Result Grid | Filter Rows:

id	user_id	image_id
1	1	1
2	2	2
NULL	NULL	NULL

4. Retrieve All Password Reset Requests

SELECT * FROM password_resets;

Result Grid | Filter Rows: | Edit: | Export/

id	user_id	reset_token	created_at	expires_at
1	1	randomtoken1	2025-02-28 09:32:34	2025-02-28 10:32:34
2	2	randomtoken2	2025-02-28 09:32:34	2025-02-28 10:32:34
NULL	NULL	NULL	NULL	NULL

5. Retrieve All Admins

SELECT * FROM admin;

Result Grid | Filter Rows: | Edit:

id	username	email	password_hash
1	admin1	admin@example.com	hashed_admin_password
NULL	NULL	NULL	NULL

1. INNER JOIN

Returns only matching records from both tables

Example: Get all images with the username of the creator

SELECT images.id, images.image_url, images.prompt, users.username FROM images INNER JOIN users ON images.user_id = users.id;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	image_url	prompt	username
1	https://example.com/image1.png	A futuristic cityscape at night	user1
2	https://example.com/image2.png	A cyberpunk warrior standing on a rooftop	user2

2. LEFT JOIN

Returns all records from the left table and matching records from the right table (NULL if no match).

Example: Get all users and their generated images (include users even if they haven't created any images)

SELECT users.id, users.username, images.image_url FROM users LEFT JOIN images ON users.id = images.user_id;

Result Grid | Filter Rows: | Ex

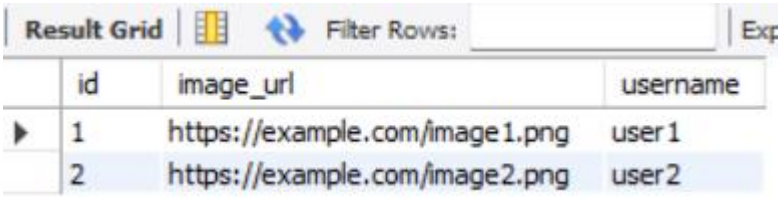
id	username	image_url
1	user1	https://example.com/image1.png
2	user2	https://example.com/image2.png

3. RIGHT JOIN

Returns all records from the right table and matching records from the left table (NULL if no match).

Example: Get all images and their creators (include images even if they are not linked to a user)

SELECT images.id, images.image_url, users.username FROM images RIGHT JOIN users ON images.user_id = users.id;



id	image_url	username
1	https://example.com/image1.png	user1
2	https://example.com/image2.png	user2

4. FULL OUTER JOIN

Returns all records when there is a match in either table (NULL where no match).

Example: Get all users and their images (including users without images and images without users)

```
SELECT users.id, users.username, images.image_url
FROM users
LEFT JOIN images ON users.id = images.user_id

UNION

SELECT users.id, users.username, images.image_url
FROM users
RIGHT JOIN images ON users.id = images.user_id;
```

Result Grid | Filter Rows:

id	username	image_url
1	user1	https://example.com/image1.png
2	user2	https://example.com/image2.png

5. CROSS JOIN

Returns the Cartesian product (combination of all rows from both tables).

Example: Get all possible user-image combinations

```
SELECT users.username, images.image_url
FROM users
CROSS JOIN images;
```

Result Grid | Filter Rows:

username	image_url
user2	https://example.com/image1.png
user1	https://example.com/image1.png
user2	https://example.com/image2.png
user1	https://example.com/image2.png